SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DESIGN AND ANALYSIS OF ALGORITHMS
PROJECT-BASED LEARNING (PBL) REPORT

PROJECT TITLE

Cloud Task Placement: Latency-Aware Replica and VM Placement using MST Backbone, Shortest Paths and Dynamic Programming

Program	B. Tech – Computer Science and Engineering
Section	Panthers
Faculty	Dr. Monday jubrin Abdullahi
Academic Year	2026 – 27
Group	Group 11

Submitted by

Roll Number	Student Name
25WU0104027	Suhaan Kapoor
25WU0101141	Sushil Pal
25WU0101142	Tanishq Hanumanta
25WU0801004	Arjun Pal

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Woxsen University for providing us with the opportunity to undertake this project as part of the Design and Analysis of Algorithms course.

We are deeply grateful to our faculty mentor, **Dr. Monday jubrin Abdullahi**, for the valuable guidance, constructive suggestions, and continuous support throughout the development of our project, “Cloud Task Placement: Latency-Aware Replica and VM Placement using MST Backbone, Shortest Paths and Dynamic Programming.” Their guidance helped us understand the practical application of algorithm design, complexity analysis, optimization, and problem-solving techniques to a real-world cloud infrastructure problem.

We would also like to thank the Department of Computer Science and Engineering for providing the academic environment, resources, and support required to complete this project successfully.

We sincerely appreciate the efforts and cooperation of all our team members of Group 11, whose contributions to algorithm development, implementation, testing, experimentation, documentation, and presentation made this project possible.

Finally, we would like to thank our parents, friends, and everyone who supported and encouraged us throughout the completion of this project.

Suhaan Kapoor
Sushil Pal
Tanishq Hanumanta
Arjun Pal

ABSTRACT

Modern cloud platforms must decide where to place service replicas and virtual machines (VMs) across a network of servers so that users experience low latency while no server is overloaded. Placing replicas without regard to the network topology, or ignoring capacity limits, produces long request paths, overloaded hosts, and wasted backbone capacity. The combined problem contains the k-median problem as a special case and is therefore computationally difficult.

This project presents **CloudPlace DAA**, a deterministic and explainable decision-support engine that applies classical Design and Analysis of Algorithms techniques to cloud task placement. Servers are modelled as nodes of a weighted graph and link latencies as edge weights. A minimum spanning tree (MST), computed with Kruskal's and Prim's algorithms, forms the low-cost network backbone. Dijkstra's algorithm computes shortest-latency paths for request routing, and Floyd–Warshall is used as an independent verification oracle. Replica placement is solved by three selection strategies: a greedy k-median heuristic, a dynamic programming (DP) formulation over the MST ordering, and an exhaustive search that serves as an exact reference for small instances. A one-swap local search is applied to refine greedy solutions, and an independent validator re-checks capacity and objective values.

Experiments on a 12-server demonstration network ($k = 3$ replicas, total demand 131 units) show that the MST backbone carries only 28.7% of the total link weight yet stretches paths by an average of 1.48×, which is why the backbone is used for provisioning while requests are routed on shortest paths. The exact optimum was 21.14 ms average latency; greedy plus local search matched it, greedy alone was 3.1% above it, and the DP over the MST ordering was 20.4% above it. Over 30 random instances, greedy plus local search reached the optimum in 27 cases with a mean gap of 0.52%, while the DP had a mean gap of 11.67%. Dijkstra was 13.5× faster than Floyd–Warshall at 300 servers. The results demonstrate that greedy methods, dynamic programming, spanning trees, shortest paths, exhaustive search, and local search can be combined into a measurable and reproducible solution, and that a method optimal for a relaxed subproblem is not automatically optimal for the complete placement problem.

TABLE OF CONTENTS

ACKNOWLEDGEMENT 2
ABSTRACT 3
TABLE OF CONTENTS 4
CHAPTER 1 – INTRODUCTION 6
1.1 Introduction 6
1.2 Problem Statement and Motivation 6
1.3 Objectives and Scope 7
CHAPTER 2 – PROBLEM DEFINITION 8
2.1 Problem Formulation 8
2.2 Constraints and Assumptions 8
2.3 Problem Decomposition 9
CHAPTER 3 – LITERATURE REVIEW 10
3.1 Algorithmic Background 10
3.2 Related Optimization Problems 10
3.3 Research Gap and Project Approach 11
CHAPTER 4 – SYSTEM DESIGN AND ARCHITECTURE 12
4.1 System Overview 12
4.2 Architecture and Optimization Pipeline 12
4.3 Data and Experiment Design 13
CHAPTER 5 – ALGORITHMS AND METHODOLOGY 14
5.1 Algorithms Used 14
5.2 Placement and Routing Methodology 15
5.3 Validation and Objective Evaluation 15
CHAPTER 6 – IMPLEMENTATION 16
6.1 Technology Stack 16
6.2 Engine Modules and Experiment Runner 16
6.3 Reproducibility and Safety Checks 16
CHAPTER 7 – RESULTS AND EXPERIMENTAL ANALYSIS 17
7.1 Experimental Setup and Demo Results 17
7.2 Benchmark Results 18
7.3 Discussion of Findings 20

CHAPTER 8 – TESTING AND VERIFICATION 21
CHAPTER 9 – LIMITATIONS AND FUTURE SCOPE 22
CHAPTER 10 – CONCLUSION 23
REFERENCES 24
APPENDIX A: ADDITIONAL SYSTEM EVIDENCE 25
APPENDIX B: INTERACTIVE VISUALISATION INTERFACE 26

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Cloud task placement is the process of deciding which servers in a distributed infrastructure should host the replicas of a service or the virtual machines of a workload. Every replica placed on a server serves a share of the user demand, and the latency experienced by each user depends on the network distance between the user's access point and the replica that serves it. Placement decisions therefore determine response time, load balance, and the amount of backbone capacity that must be provisioned.

The problem becomes challenging when servers have different demand levels and capacities, links have different latencies, and only a limited number of replicas can be deployed. Placing replicas on the busiest servers alone may leave distant clients poorly served, while placing them purely by geometric spread may overload some hosts.

The CloudPlace DAA project addresses this problem using classical algorithms from Design and Analysis of Algorithms. The system combines graph algorithms, minimum spanning trees, shortest paths, greedy algorithms, dynamic programming, exhaustive search, local search, and constraint validation to produce and evaluate replica placements.

1.2 Problem Statement and Motivation

The project considers a set of servers connected by links of known latency. Each server generates client demand and has a hosting capacity. The task is to select k servers to host replicas and to assign every server's demand to a replica so that the demand-weighted average latency is minimised without exceeding any replica's capacity.

In addition to placement, the network must be provisioned: a minimum spanning tree is computed as a low-cost backbone, and shortest paths are computed for request routing. The placement stage selects replicas using greedy heuristics, dynamic programming, or exact enumeration.

The complete problem is computationally difficult because it contains the k-median problem as a special case, and the capacity constraints make it harder still. Solving it optimally for arbitrary input sizes is therefore not practical. The project is motivated by the opportunity to apply DAA concepts to a realistic infrastructure problem and to experimentally compare exact, approximate, and heuristic algorithms. Important concepts demonstrated include:

- Minimum spanning trees (Kruskal and Prim)
- Shortest-path algorithms (Dijkstra and Floyd–Warshall)
- Dynamic programming
- Greedy algorithms
- Local search and approximation
- Exhaustive search
- Complexity analysis

A major motivation is also to demonstrate that the algorithm that is optimal for one stage of a problem, or for a simplified version of it, does not necessarily produce the best result for the complete system.

1.3 Objectives and Scope

The main objectives of the project are to model the server network as a weighted graph, compute a minimum spanning tree backbone, calculate shortest-latency paths, implement different placement techniques, assign client demand under capacity limits, independently validate every solution, and compare all algorithms fairly on identical inputs.

The system also aims to verify the heuristic and DP solutions against an exhaustive-search optimum, to verify Dijkstra against Floyd–Warshall and Kruskal against Prim, and to measure how running time and solution quality scale with the number of servers.

The scope includes backbone construction, shortest-path routing, replica placement, capacity-constrained assignment, local search improvement, algorithm comparison, and benchmarking. The project does not use machine learning, artificial intelligence, or black-box optimization solvers. All decisions are produced by deterministic algorithms implemented within the system. It also does not claim to solve the capacitated k-median problem optimally for large inputs.

CHAPTER 2 – PROBLEM DEFINITION

2.1 Problem Formulation

The server network is represented as an undirected weighted graph:

$$G = (V, E)$$

where V is the set of servers, E is the set of links, and w(u, v) is the positive latency of link (u, v) in milliseconds. The shortest-path latency between two servers is denoted dist(u, v).

Each server i in V has:

- a demand d_i (integer units of client load generated at that server);
- a capacity c_i (the maximum demand it can serve if it hosts a replica).

A placement is a set R of exactly k servers chosen from V, together with an assignment a that maps every server to one of them. The objective is to minimise the demand-weighted average latency:

$$L(R, a) = \Sigma_i \, d_i \cdot dist(i, a(i)) \, / \, \Sigma_i \, d_i$$

The solution must satisfy:

- exactly k replicas are placed, |R| = k;
- every server's demand is assigned to exactly one replica;
- replica load within capacity, $\Sigma_{i \,:\, a(i)=r} \, d_i \leq c_r$ for every replica r in R;
- latency measured along shortest paths in G.

The network backbone is the minimum spanning tree T of G, whose total cost is the sum of w(e) over all edges e of T. These inputs and constraints form the mathematical basis of the CloudPlace system.

2.2 Constraints and Assumptions

The system assumes that demand and capacity values are integers, and that latency is measured in milliseconds. Link latencies are static and symmetric, and the graph is connected so that every server can reach every replica. A replica serves the demand of its own server at zero network latency.

The current implementation uses one capacity dimension, load units, and does not model memory, storage, or bandwidth separately. The number of replicas k is given as an input. Hosting cost is not part of the objective in the current implementation and is listed as future work. Capacity is enforced during assignment, so a placement whose replicas cannot absorb the total demand is reported as infeasible rather than silently overloaded.

2.3 Problem Decomposition

Rather than attempting to solve the complete NP-hard problem directly, CloudPlace divides it into stages.

First, Dijkstra's algorithm computes the all-pairs shortest-latency matrix, and Kruskal's or Prim's algorithm computes the MST backbone. Next, a placement algorithm (greedy k-median, DP over the MST ordering, or exhaustive search) selects k replica servers. A capacity-aware assignment stage then routes each server's demand to the nearest replica that still has room. An optional one-swap local-search stage improves the placement.

Finally, an independent validator re-checks capacities and recomputes the objective. For small instances, exhaustive search is also available to compare every heuristic against the true optimum.

CHAPTER 3 – LITERATURE REVIEW

3.1 Algorithmic Background

The project is based on several fundamental algorithmic concepts. Choosing k facilities to minimise the total weighted distance to clients is the classical p-median problem, introduced for networks by Hakimi (1964) and shown to be NP-hard in general graphs (Kariv & Hakimi, 1979; Garey & Johnson, 1979).

Minimum spanning trees are computed with Kruskal's algorithm, which sorts edges and adds the cheapest edge that does not form a cycle, and with Prim's algorithm, which grows a tree from a root using a priority queue. The cut property guarantees that both produce a spanning tree of minimum total weight. Union-Find (Tarjan, 1975) makes the cycle test in Kruskal's algorithm almost constant time.

Dijkstra's algorithm is used to determine shortest-latency paths from every server. Floyd–Warshall is also implemented to calculate all-pairs shortest paths and is used as a verification method for Dijkstra.

Dynamic programming (Bellman, 1957) solves problems with overlapping subproblems and optimal substructure. Greedy algorithms generate solutions quickly by repeatedly taking the locally best step, but are not generally optimal for k-median. Exhaustive search is used as a correctness oracle for small instances. The project also uses one-swap local search, for which constant-factor approximation guarantees are known for the k-median problem (Arya et al., 2004).

3.2 Related Optimization Problems

The problem is related to several established optimization problems. The k-median problem represents the replica-selection component. The capacitated facility location problem adds the capacity constraint. Web-replica placement on the Internet has been studied with greedy algorithms that repeatedly add the replica giving the largest reduction in client latency (Qiu et al., 2001), which is the strategy used as the greedy baseline in this project.

The backbone component is the minimum spanning tree problem, and the routing component is the single-source shortest-path problem. Together these form a pipeline in which a cheap provisioning structure (the MST) and a low-latency routing structure (shortest paths) serve different purposes. These problems are computationally difficult in combination, which

motivates the use of exact algorithms for small subproblems and heuristics for larger instances.

3.3 Research Gap and Project Approach

A key observation in this project is that a solution that is optimal for a simplified version of the problem is not necessarily optimal for the full problem. The dynamic programming algorithm used here is exact when servers are served in contiguous segments of the MST ordering and capacity is ignored, but real assignment is capacity-constrained and the MST is not a shortest-path structure. Consequently, a placement that the DP considers optimal can be significantly worse after capacity-aware assignment on the real latency matrix.

CloudPlace addresses this issue by combining placement algorithms with a capacity-aware assignment stage and evaluating every complete solution using true shortest-path latencies. The project therefore focuses not only on algorithmic optimality for individual components but also on the relationship between the components.

CHAPTER 4 – SYSTEM DESIGN AND ARCHITECTURE

4.1 System Overview

CloudPlace is implemented as an algorithm engine with a reproducible experiment runner and a report-figure generator. The engine contains the graph generator, shortest-path and spanning-tree algorithms, placement algorithms, local search, the capacity-aware assignment routine, and the validator. It is separated from the experiment runner, which allows the algorithms to be tested and benchmarked independently.

4.2 Architecture and Optimization Pipeline

The system follows the flow shown in Figure 1: latency graph → Dijkstra all-pairs matrix → MST backbone → placement algorithm → capacity-aware assignment with local search → validation and objective evaluation.

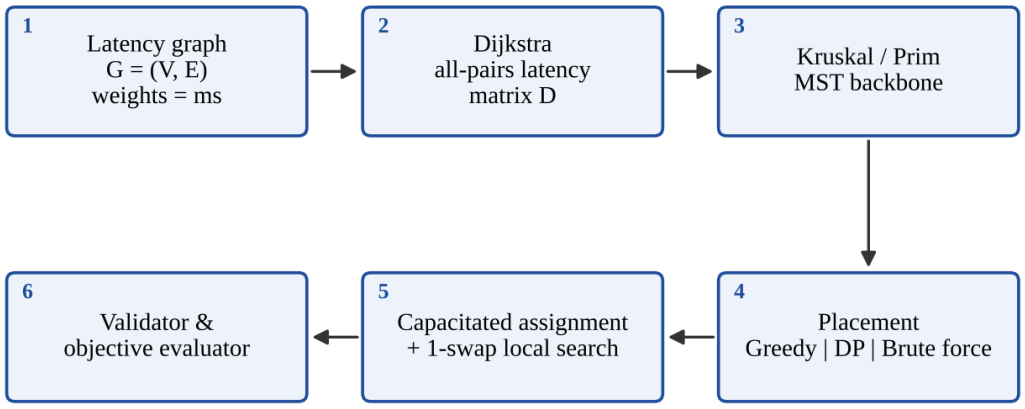


Figure 1 – CloudPlace optimization pipeline

The pipeline begins with a frozen problem instance. Shortest-path information is calculated, the MST backbone is built, and one of the available placement algorithms is applied. The selected replicas are then assigned demand, optionally improved by local search, independently validated, and finally evaluated for average latency. The same input instance and latency matrix are used when comparing different algorithms.

4.3 Data and Experiment Design

Experiments use a synthetic generator with fixed random seeds. Servers are placed uniformly at random in a 100 × 100 plane. A random spanning tree guarantees connectivity, after which 1.5n additional random links are added. Each link latency is 0.8 times the Euclidean distance plus a random 1–5 ms processing term. Demand is drawn uniformly from 1–20 units per server, and capacities are scaled so that the k replicas together can absorb about 1.7 times the total demand, with ±30% variation between servers.

Results are stored in a JSON file and every table and figure in this report is generated directly from it, so the reported numbers can be reproduced exactly.

CHAPTER 5 – ALGORITHMS AND METHODOLOGY

5.1 Algorithms Used

The project implements a collection of algorithms, each serving a specific purpose.

Dijkstra's algorithm calculates shortest-latency paths from a source server. With a binary heap its complexity is $O((V + E) \log V)$ per source, and running it from every server costs $O(V(V + E) \log V)$.

Floyd–Warshall calculates all-pairs shortest paths in $O(V^3)$ and is primarily used as a verification oracle.

Kruskal's algorithm sorts the edges and uses Union-Find to add the cheapest edge that connects two different components, in $O(E \log E)$. **Prim's algorithm** grows the tree from server 0 using a priority queue in $O(E \log V)$. Both return a minimum spanning tree of the same total weight, which is checked in testing.

Greedy k-median starts with no replicas and, in each of k rounds, adds the server that most reduces the demand-weighted latency of the current set. Its complexity is $O(k^2n^2)$.

Dynamic programming over the MST ordering first linearises the backbone by a depth-first traversal of the MST. It then partitions this ordering into k contiguous segments, each served by the median server of its segment. The state is:

dp[i][j] = least cost of serving the first i servers using j replicas

with the recurrence $dp[i][j] = \min_{p<i} (dp[p][j-1] + cost(p, i-1))$, where cost(a, b) is the best single-median cost of the segment a…b. The recurrence takes $O(n^2k)$ time after the segment costs are cached, and the segment costs are computed in $O(n^4)$ in the worst case. Backtracking recovers the chosen replicas.

Exhaustive search evaluates all C(n, k) replica subsets with the capacity-aware assignment and is used only for small instances to verify the other methods.

One-swap local search repeatedly replaces one replica with one non-replica server whenever this lowers the capacity-aware average latency, until no swap improves the solution. It never increases latency.

5.2 Placement and Routing Methodology

The placement stage and the assignment stage are intentionally separated. The placement algorithms decide which k servers host replicas, using the latency matrix and demand. The assignment stage then determines whether the demand can actually be served within capacity.

The capacity-aware assignment considers servers in order of decreasing demand and sends each one to the nearest replica that still has enough remaining capacity. If no replica has room, the placement is reported as infeasible. The MST backbone is used for provisioning cost and as a topology reference, while client requests are routed over shortest paths, because routing over the MST alone stretches paths considerably (Chapter 7).

5.3 Validation and Objective Evaluation

After assignment, the system independently recomputes every replica's load and the average latency from the latency matrix. It checks that the number of replicas equals k, that each server is assigned exactly once, that each load is within capacity, and that the recomputed objective equals the reported one.

The final objective is calculated as the demand-weighted average latency:

$$AvgLatency = \Sigma\ demand \times dist(server, replica) / \Sigma\ demand$$

This independent validation prevents a placement routine from incorrectly reporting an invalid plan as successful.

CHAPTER 6 – IMPLEMENTATION

6.1 Technology Stack

The algorithm engine is implemented in Python 3 using only the standard library (heapq, itertools, math, random, time) for the algorithms themselves. Figures are produced with Matplotlib, and this report is generated programmatically as a PDF with ReportLab.

6.2 Engine Modules and Experiment Runner

The engine module provides the random connected graph generator, Dijkstra, Floyd–Warshall, Union-Find with Kruskal, Prim, the depth-first MST ordering, the three placement algorithms, one-swap local search, and the capacity-aware assignment routine.

The experiment runner builds the demonstration scenario, runs every algorithm on identical inputs, repeats the comparison on 30 random instances, measures running times for increasing network sizes, and stores everything in a results file. A separate figure script reads this file and draws the network, charts, and pipeline diagram.

6.3 Reproducibility and Safety Checks

The project uses seeded randomness, deterministic tie-breaking, and a single stored results file to ensure reproducibility. Assignment never silently overloads a server: an infeasible placement is returned as infeasible. Because all algorithms use the same latency matrix, differences in results come from the algorithms rather than the inputs.

Timings are measured in pure Python on a single run on the development machine, so absolute values are indicative; the relative behaviour and growth rates are the meaningful outcome.

The interactive Replica Placement Lab, a browser-based visualisation of the same pipeline, is shown in Appendix B (Figures 8–11).

CHAPTER 7 – RESULTS AND EXPERIMENTAL ANALYSIS

7.1 Experimental Setup and Demo Results

The main demonstration uses a 12-server network with 25 links, a total client demand of 131 units, and k = 3 replicas. Per-server demand and capacity are listed in Appendix A.

The MST backbone has a total latency weight of 305.16 ms, identical for Kruskal and Prim, compared with 1064.98 ms for all links, so the backbone needs only 28.7% of the total link weight. Routing every request over the MST alone would, however, lengthen shortest paths by 1.48× on average and by up to 4.13× in the worst case, and 27 of the 66 server pairs happen to keep their shortest path inside the tree.

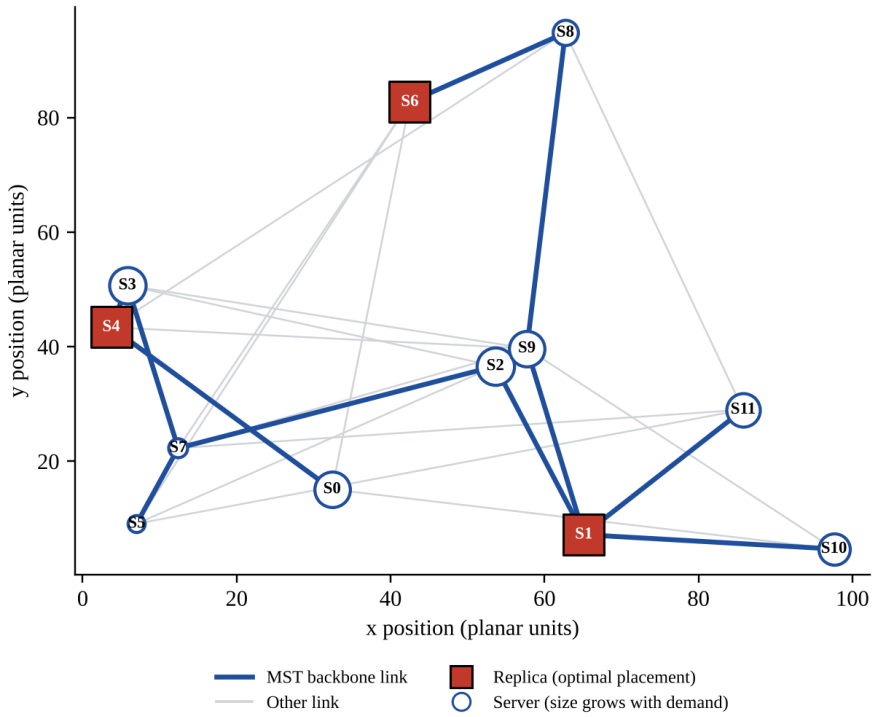


Figure 2 – Demonstration network, MST backbone and optimal replica placement

Exhaustive enumeration of all C(12, 3) = 220 placements finds an optimal average latency of 21.14 ms with replicas on S1, S4 and S6. Table 1 compares the algorithms on identical inputs.

Algorithm	Replicas	Avg latency (ms)	Gap to optimum	Time (ms)
Brute force (exact)	S1, S4, S6	21.14	0.0% (optimal)	2.89
Greedy + local search	S1, S4, S6	21.14	0.0% (optimal)	0.90
Greedy k-median	S1, S3, S6	21.80	+3.1%	0.21
DP on MST order	S2, S4, S9	25.46	+20.4%	0.55
Random (mean of feasible)	–	33.52	+58.6%	–

Table 1 – Comparison of placement algorithms on the demonstration network

All plans were feasible with respect to capacity. Greedy plus local search found the same placement as the exhaustive search, greedy alone was 3.1% above the optimum, and the DP over the MST ordering was 20.4% above it. A random placement averaged 33.52 ms, which is 58.6% above the optimum, showing the value of all informed methods.

7.2 Benchmark Results

The benchmark experiments confirmed the expected algorithmic behaviour.

For shortest paths, running Dijkstra from every server took 2.5 ms at 50 servers and 61.2 ms at 300 servers, whereas Floyd–Warshall took 5.3 ms and 824.0 ms, so Dijkstra was 13.5× faster at 300 servers (Figure 3). Kruskal and Prim both stayed below 1 ms up to 300 servers.

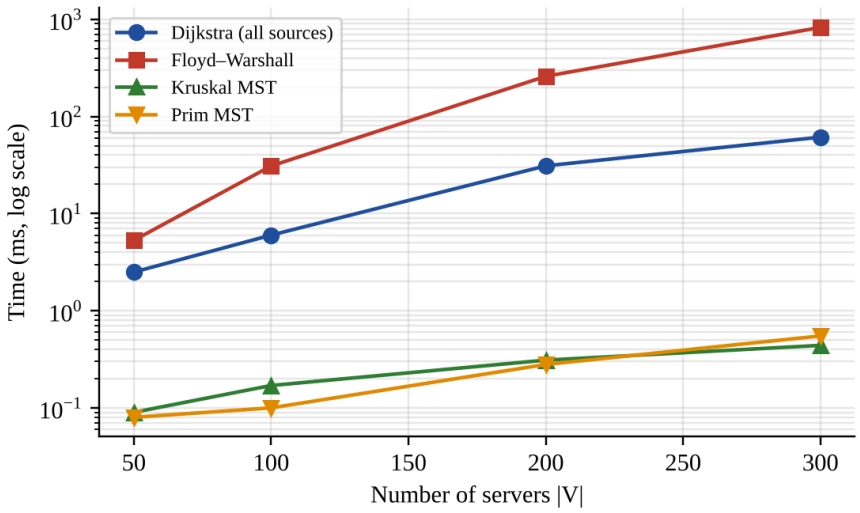


Figure 3 – Running time of shortest-path and spanning-tree algorithms

For placement with k = 3, exhaustive search took 0.7 ms at 10 servers and 4.5 ms at 16 servers, because it evaluates C(n, 3) subsets; the DP took 0.68 ms and greedy 0.189 ms at 16 servers (Figure 4). With k = 4, the DP grew from 22.0 ms at 40 servers to 1.73 s at 120

servers, consistent with its high-order polynomial cost, while greedy remained at 29.2 ms.

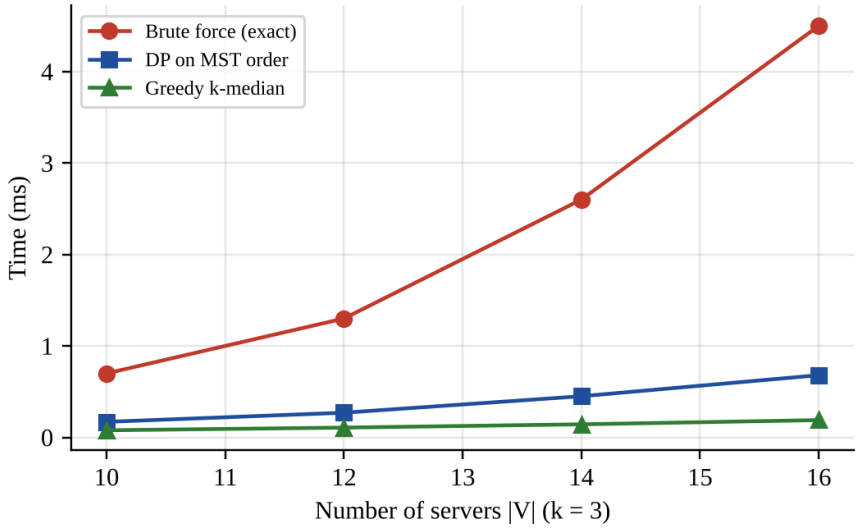


Figure 4 – Placement algorithm running time against network size (k = 3)

Adding replicas gives diminishing returns. On the demonstration network the greedy average latency fell from 47.4 ms with one replica to 30.27 ms, 21.8 ms, 16.91 ms, 12.3 ms, and 8.91 ms with two to six replicas (Figure 5), so the first few replicas deliver most of the benefit.

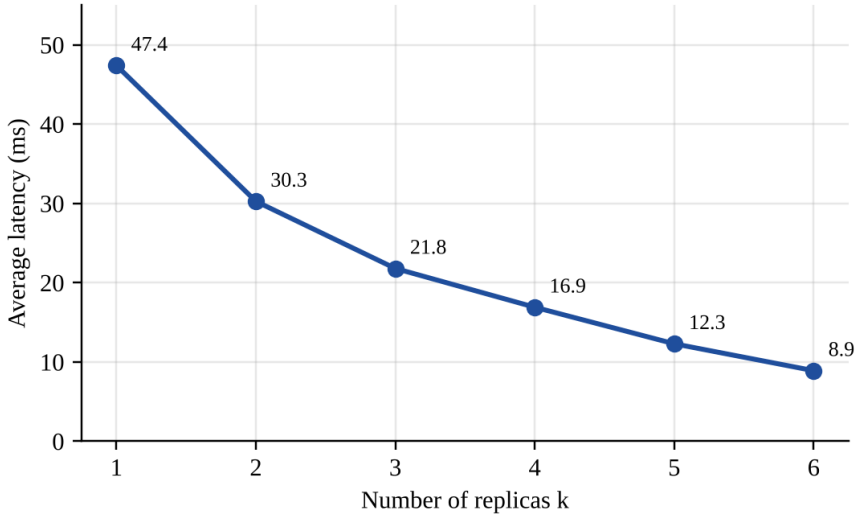


Figure 5 – Greedy average latency against number of replicas k

Placement quality was measured against the exhaustive optimum on 30 random instances of 11 servers with k = 3 (Table 2, Figure 6).

Algorithm	Mean gap	Worst gap	Optimal in
DP on MST order	11.67%	50.84%	15 of 30
Greedy k-median	9.86%	44.06%	9 of 30

Algorithm	Mean gap	Worst gap	Optimal in
Greedy + local search	0.52%	10.91%	27 of 30

Table 2 – Gap to the exact optimum over 30 random instances

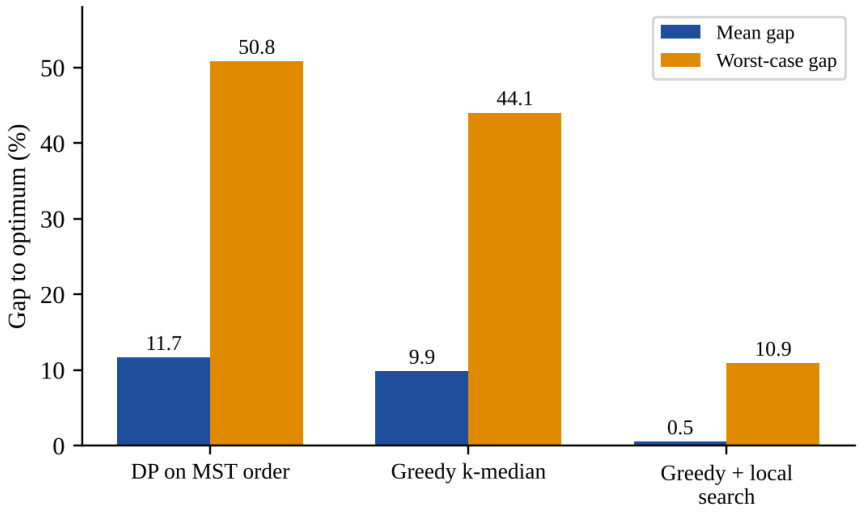


Figure 6 – Mean and worst-case optimality gap of the placement algorithms

7.3 Discussion of Findings

The experiments lead to several important findings.

First, theoretical complexity is reflected in measured performance: Dijkstra scales much better than Floyd–Warshall, exhaustive search grows combinatorially with the number of servers, the greedy method grows polynomially and slowly, and the DP is exact for its segment model but costly for large networks.

Second, the MST is the right structure for backbone provisioning, since it needs under 30% of the total link weight, but not for routing, since it lengthens average paths by about 48%. Shortest paths should carry the requests.

Third, the DP over the MST ordering is optimal for its contiguous-segment relaxation but not for the complete capacitated problem, with a mean gap of 11.67% and a worst gap of 50.84%. A simple greedy placement followed by one-swap local search reached the optimum in 27 of 30 instances with a mean gap of only 0.52%.

Finally, the exact search provides a useful benchmark for small instances. The project's central finding is therefore that optimizing a subproblem independently does not guarantee optimization of the complete system.

CHAPTER 8 – TESTING AND VERIFICATION

The project uses several levels of verification to check both the individual algorithms and the complete pipeline.

Cross-algorithm checks compare independent implementations of the same quantity:

- Dijkstra's all-pairs latencies equal the Floyd–Warshall matrix on the demonstration network and on all 30 random instances;
- Kruskal's and Prim's MSTs have identical total weight (305.16 ms on the demonstration network) on the demonstration network and on all 30 random instances.

Oracle-based checks compare heuristics with exhaustive search over 30 random instances of 11 servers with k = 3:

- no heuristic ever reported a latency below the exhaustive optimum (0 violations);
- one-swap local search never returned a placement worse than its greedy starting point (0 cases);
- every reported placement was feasible with respect to capacity.

An independent validator recomputes replica loads and the average latency from scratch for every heuristic placement and confirms that no capacity is exceeded and that the recomputed latency matches the reported value. Seeded generation and a stored results file allow every check to be repeated exactly.

These checks were performed by the experiment scripts of this project. No claim is made about a larger automated test suite or about formal proofs of the heuristics beyond the cited theory.

CHAPTER 9 – LIMITATIONS AND FUTURE SCOPE

The current placement methods are heuristic and do not guarantee the globally optimal capacitated placement for large instances. The local-search stage reaches a local optimum, and the greedy assignment of demand to the nearest replica with room is itself a heuristic.

The DP is exact only for its contiguous-segment relaxation over the MST ordering, and its segment-cost computation limits it to networks of a few hundred servers. The system also uses only a single load dimension as capacity, assumes static symmetric latencies, treats the number of replicas as given, and omits hosting cost from the objective.

The experiments use synthetic planar networks rather than measurements from a real data center or wide-area network, and timings come from single runs in pure Python.

Future development can include hosting-cost and budget constraints, choosing k automatically, multi-dimensional resource constraints (CPU, memory, bandwidth), a fully capacity-aware DP or integer-programming formulation, tree-based exact DP on the MST, stronger metaheuristics such as simulated annealing and tabu search, time-varying demand and live latency measurements, fault-tolerant placement with replica failures, and deployment as a service with a web interface.

CHAPTER 10 – CONCLUSION

CloudPlace DAA demonstrates how classical algorithmic techniques can be applied to a realistic cloud task placement problem involving replica selection, server capacity, and network latency.

The project combines minimum spanning trees, shortest-path algorithms, greedy heuristics, dynamic programming, local search, exhaustive verification, and independent constraint validation. The decomposition allows each part of the problem to be addressed using an appropriate algorithm while keeping the system explainable and reproducible.

The experimental results demonstrate the difference between optimizing a simplified component and optimizing the complete placement. Although the dynamic programming method is optimal for its backbone-ordering model and runs quickly on small networks, its placement on the demonstration network had an average latency of 25.46 ms against the exact optimum of 21.14 ms. In comparison, greedy selection followed by one-swap local search matched the optimum, and over 30 random instances it reached the optimum 27 times with a mean gap of 0.52%.

The benchmark and verification results further demonstrate that the implemented algorithms behave consistently with their theoretical properties: Dijkstra outperforms Floyd–Warshall on sparse graphs, Kruskal and Prim agree, and exhaustive search grows combinatorially. Overall, the project shows that effective algorithm design is not simply about finding an optimal solution to one subproblem, but about understanding how different algorithmic stages interact to produce the final system-level result.

REFERENCES

Arya, V., Garg, N., Khandekar, R., Meyerson, A., Munagala, K., & Pandit, V. (2004). Local search heuristics for k-median and facility location problems. *SIAM Journal on Computing, 33*(3), 544–562.

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271. https://doi.org/10.1007/BF01386390

Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM, 5*(6), 345. https://doi.org/10.1145/367766.368168

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.

Hakimi, S. L. (1964). Optimum locations of switching centers and the absolute centers and medians of a graph. *Operations Research, 12*(3), 450–459.

Kariv, O., & Hakimi, S. L. (1979). An algorithmic approach to network location problems. II: The p-medians. *SIAM Journal on Applied Mathematics, 37*(3), 539–560.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society, 7*(1), 48–50.

Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal, 36*(6), 1389–1401.

Qiu, L., Padmanabhan, V. N., & Voelker, G. M. (2001). On the placement of web server replicas. *Proceedings of IEEE INFOCOM 2001*, 1587–1596.

Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM, 22*(2), 215–225. https://doi.org/10.1145/321879.321884

Warshall, S. (1962). A theorem on Boolean matrices. *Journal of the ACM, 9*(1), 11–12. https://doi.org/10.1145/321105.321107

APPENDIX

APPENDIX A: ADDITIONAL SYSTEM EVIDENCE

1. The following table lists the demand and capacity of every server in the demonstration scenario used in Chapter 7.

Server	Demand	Capacity	Server	Demand	Capacity
S0	16	72	S6	9	58
S1	1	81	S7	3	64
S2	18	82	S8	7	73
S3	17	85	S9	16	55
S4	16	64	S10	12	67
S5	2	73	S11	14	88

Table A1 – Demand and capacity of the demonstration servers

2. The following table shows the assignment produced for the optimal placement and confirms that no replica exceeds its capacity.

Replica	Servers served	Load	Capacity	Utilisation
S1	S1, S2, S9, S10, S11	61	81	75%
S4	S0, S3, S4, S5, S7	54	64	84%
S6	S6, S8	16	58	28%

Table A2 – Assignment and capacity utilisation of the optimal placement

3. The following figure shows how much longer paths become when requests are forced to use only the MST backbone. Most server pairs stay close to their shortest path, but a few pairs are stretched by more than three times, which is why shortest paths are used for routing.

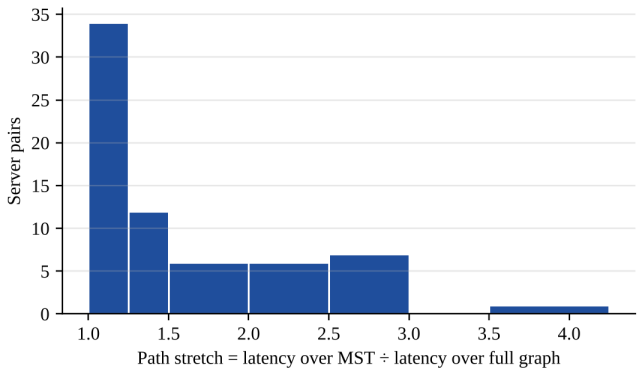


Figure 7 – Distribution of path stretch when routing over the MST backbone

APPENDIX B: INTERACTIVE VISUALISATION INTERFACE

1. The following figures show the Replica Placement Lab, an interactive visualisation prototype that accompanies the CloudPlace engine. It draws the server network on a rotatable 3D globe and lets the user change the number of servers and replicas, generate a new random topology, and inspect the route and replica assigned to any server. The scenarios shown here are randomly generated topologies with 10 servers and k = 3, so their values differ from the 12-server demonstration network analysed in Chapter 7.



Figure 8 – Replica Placement Lab: globe view with the metrics panel, controls and server table

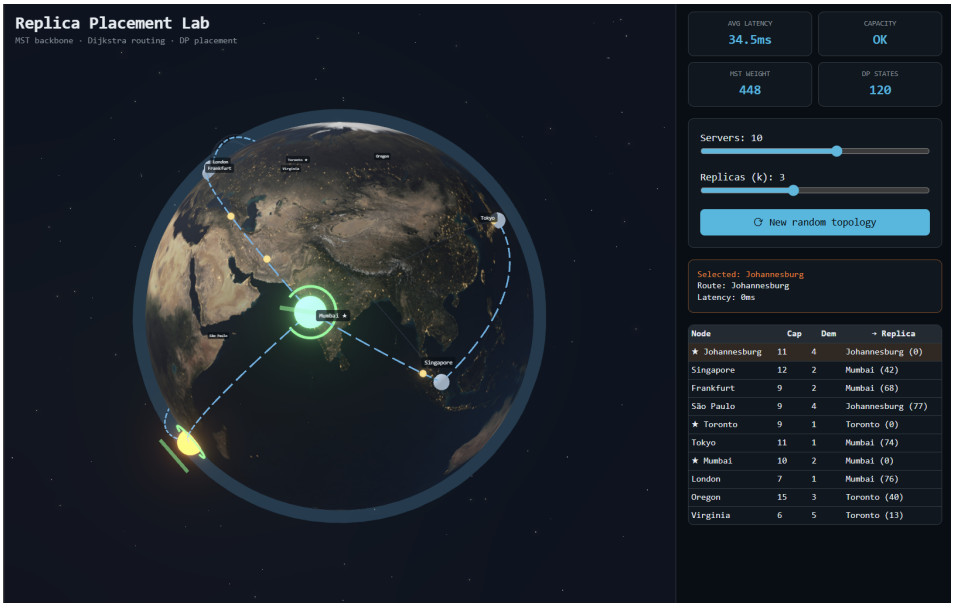


Figure 9 – The same topology after rotating the globe to a different viewing angle

2. The side panel summarises the current scenario. The metric tiles report the average latency, the capacity status, the MST weight, and the number of DP states. The sliders set the number of servers and the number of replicas k, and the selected server is shown with its route and latency. The table lists the capacity and demand of every server together with the replica it is assigned to; replica servers are marked with a star.



Node	Cap	Dem	→ Replica
★ Johannesburg	11	4	Johannesburg (0)
Singapore	12	2	Mumbai (42)
Frankfurt	9	2	Mumbai (68)
São Paulo	9	4	Johannesburg (77)
★ Toronto	9	1	Toronto (0)
Tokyo	11	1	Mumbai (74)
★ Mumbai	10	2	Mumbai (0)
London	7	1	Mumbai (76)
Oregon	15	3	Toronto (40)
Virginia	6	5	Toronto (13)

Figure 10 – (a) Metrics and control panel; (b) per-server capacity, demand and assigned replica

3. The globe can be rotated by dragging, and a server is selected by tapping it. Replica servers are highlighted and linked by dashed lines, and the same network can be shown as a wireframe globe or on a textured Earth.

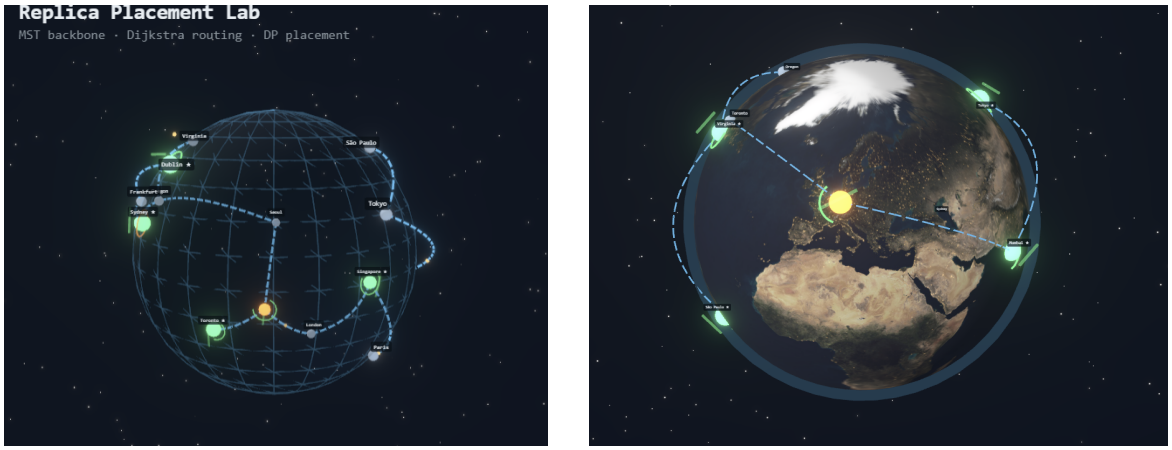


Figure 11 – (a) Wireframe globe view; (b) close-up of a replica placement on the textured globe